Here is the complete documentation and analysis based on the provided Examination Result Python script.

1. Problem Statement, Objectives, and Assumptions

Problem Statement

Evaluating academic performance across multiple subjects and applying conditional grace marks policies manually can be tedious and prone to calculation errors. An automated command-line tool is required to collect student subject marks, validate input ranges, apply a controlled grace mark policy to near-passing scores, and compute the final aggregate result.

Objectives

- Prompt for and collect marks across five academic subjects.
- Validate that all entered marks fall within the valid range of 0 to 100.
- Apply a maximum grace pool of 5 marks to boost any individual score falling between 30 and 34 (inclusive) up to the passing mark of 35.
- Calculate the total aggregate score out of 500 and the average percentage.
- Determine and display the final pass or fail status based on whether all final marks meet the minimum threshold of 35.

- ### Assumptions
- The curriculum consists of exactly five subjects, each graded out of a maximum of 100 marks.
-
- The passing threshold for each subject is 35.
-
- Grace marks are allocated dynamically from a pool of 5 points to the first eligible subjects encountered.
-

2. Algorithm, Flowchart, and Pseudocode

Algorithm

1. Initialize an empty list called marks.
2.
3. Run a loop 5 times to prompt the user for each subject's mark and append it to marks.
4.
5. Validate all marks: check if any mark is less than 0 or greater than 100.
6.
7. If invalid, display an error message and terminate the evaluation.
8.
9. If valid, create a copy of the marks list called final_marks and set the grace pool to 5.
10.

11.

    Iterate through final_marks: if a mark is between 30 and 34, calculate the needed points to reach 35. If needed is less than or equal to the available grace, apply the boost, reduce the remaining grace, and update the score.

12.

13.

    Calculate the total sum and average percentage of the adjusted marks.

14.

15.

    Check if all final marks are greater than or equal to 35. If true, print "Result: PASS"; otherwise, print "Result: FAIL".

16.

    Flowchart Representation

    Pseudocode

3. Python Source Code and Test Cases

Python Source Code

```
            for   in range 5
             int input "Enter mark for subject "    str     1     ": "


        Truefor         in
   if           0 or           100
                False
 if not
          "Invalid marks: each mark must be 0 to 100." else
            5


   for   in range 5
      if 30                      35
                    35
          if



            sum
                       5
          "Final marks:"
          "Total:"          "/ 500"
          "Average:"  round            2   "%"

   if all             35 for         in
             "Result: PASS"
   else
             "Result: FAIL"
```

(Source code adapted from).

Test Cases & Sample Output

Test Case ID	Scenario Description	User Inputs	Expected Output / Behavior
TC-01	All Passing / High Scores	80, 75, 90, 85, 88	Final marks: unchanged. Total: 418 / 500. Result: PASS.
TC-02	Grace Marks Applied Successfully	32, 70, 65, 80, 75	First subject bumped from 32 to 35 using grace. Result: PASS.
TC-03	Failure Due to Low Score Beyond Grace	28, 50, 60, 70, 80	First subject remains 28 (outside grace range of 30-34). Result: FAIL.
TC-04	Invalid Mark Range	105, 50, 60, 70, 80	"Invalid marks: each mark must be 0 to 100.".

Simulated Terminal Output (for TC-02)

4. Conclusion

The examination result script provides a robust and fair mechanism for handling student scorecards. By incorporating strict input validations, a conditional grace marks allocation workflow, and final aggregate performance checks, it automates academic evaluation efficiently.